OPERATOR PADA BAHASA C

FEBRIAN WINSTON HUTAGALUNG, S.T., M.T.

Outline

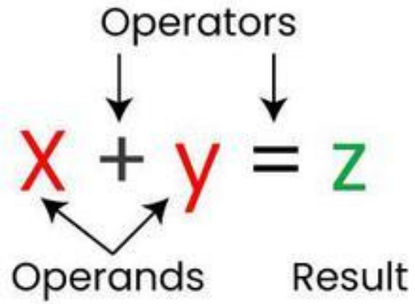
1. Operator
2. Assignment Operator
3. Arithmetic Operator
4. Relational and Logical Operator
5. Bitwise Operator
6. Special Operator

Operator

What is Operator?

- Operator adalah simbol khusus dalam pemrograman untuk melakukan operasi pada data (variabel, nilai).
- Fungsinya mirip alat hitung atau alat logika.

Operators in Programming



Operator

There are four main classes of operators:

- Arithmetic
- Relational
- Logical
- Bitwise

In addition, there are some special operators, such as the assignment operator, for particular tasks.

Assignment Operator

- dipakai setiap kali menyimpan data (menyimpan nilai input)
- = (single equal sign)
- general form:

  variable_name = expression;

- Assignment may involve type conversion: The value of the right side (expression side) of the assignment is converted to the type of the left side (target variable)

example:

```
int umur = 20;    // simpan umur mahasiswa
```

Assignment Operator

Multiple Assignment

- You can assign many variables the same value by using multiple assignments in a single statement.
- Example below is assigned 0 to variable x, y, and z on the nearly same time:

example:

```
x = y = z = 0
```

Assignment Operator

Compound Assignment

- There is a variation on the assignment statement, called compound assignment, that simplifies the coding of a certain type of assignment operations.
- for example:

```
x = x + 10
```

- can be written as:

```
x += 10
```

Assignment Operator

Compound Assignment

- example:

```
int x = 5;
x += 10; // atau x = x + 10
printf("%d", x);
```

output: 15

```
x -= 5;
x *= 2;
x /= 3
x %= 4;
```

Arithmetic Operator

Operator	Meaning	Example
–	Subtraction, also unary minus	5 – 3 = 2
+	Addition	10+7 = 17
*	Multiplication	2 * 4 = 8
/	Division	6 / 3 = 2
%	Modulus	18 % 5 = 3
––	Decrement	
++	Increment	

Arithmetic Operator

```c
#include <stdio.h>
int main()
{
    int x,y;
    x=10;
    y=20;

    printf("%d * %d = %d\n",x,y,x*y);
    printf("%d + %d = %d\n",x,y,x+y);
    printf("%d - %d = %d\n",x,y,x-y);
    printf("%d / %d = %f\n",x,y,(float)x/y);
    printf("%d mod %d = %d\n",x,y,x%y);
}
```

Keluarannya:

```
10 * 20 = 200
10 + 20 = 30
10 - 20 = -10
10 / 20 = 0.500000
10 mod 20 = 10
```

Arithmetic Operator

Increment and Decrement Operator

- These are the increment ++ and decrement -- operators.
- The operator ++ adds 1 to its operand, and – – subtracts 1.

Dua bentuk penulisan:

Postfix	Prefix	Meaning
x++;	++x;	x=x+1
x--;	--x;	x=x-1

Arithmetic Operator

Increment and Decrement Operator

- **Prefix (++x, --x) : ubah dulu, baru dipakai.**
- **example:**

```
int x = 10;
int y = ++x;
```

Prefix

```
x=10;
y=++x;
```

equivalence with

```
x=10;
x=x+1;
y=x;
```

prosesnya:

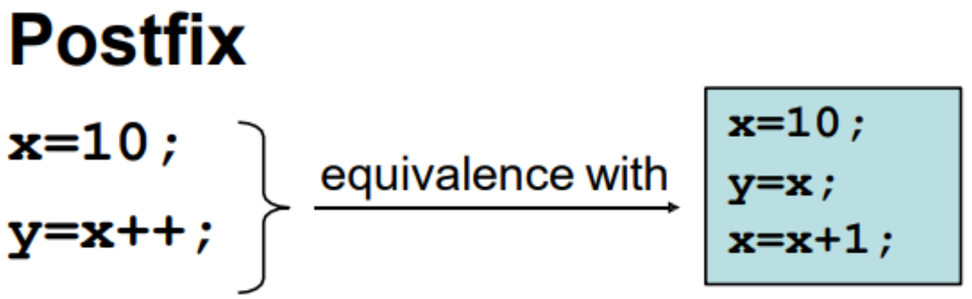
1. **++x → x dinaikkan dulu: dari 10 jadi 11**
2. **baru hasilnya dipakai untuk y**
3. **hasilnya: x = 11, y = 11**

Arithmetic Operator

Increment and Decrement Operator

- **Postfix (x++, x--) : pakai dulu, baru diubah.**
- **example:**

```
int x = 10;
int y = x++;
```



prosesnya:

1. **x++ → nilai x (10) dipakai dulu untuk y**
2. **setelah itu baru x dinaikkan jadi 11**
3. **hasilnya: x = 11, y = 10**

Relational & Logical Operator

Relational Operator	
Operator	**Meaning**
>	Greater than
>=	Greater than or equal
<	Less than
<=	Less than or equal
==	Equal
!=	Not equal

Logical Operator	
Operator	**Meaning**
&&	AND
||	OR
!	NOT

Relational & Logical Operator

Logical Operator && (AND)

Operasi	Hasil
true && true	true
true && false	false
false && true	false
false && false	false

Relational & Logical Operator

Logical Operator || (OR)

Operasi	Hasil
true || true	true
true || false	true
false || true	true
false || false	false

Relational & Logical Operator

Logical Operator ! (NOT)

Operasi	Hasil
!false	true
!true	false

Bitwise Operator

Operator	Meaning	Example
&	AND	27 & 21 = 17
|	OR	10 | 7 = 15
^	Exclusive OR (XOR)	10 ^ 29 = 23
~	One's complement (NOT)	~ $(0101)_2$ = 1010
>>	Shift Right	7 >> 2 = 1
<<	Shift Left	9 << 3 = 72

Bitwise Operator

AND (&)

hasil 1 hanya kalau keduanya 1

```
27 = 11011
21 = 10101
-----------AND
     10001 = 17
```

OR (|)

hasil 1 kalau salah satu bit bernilai 1

```
10 = 1010
 7 = 0111
-----------OR
     1111 = 15
```

Bitwise Operator

XOR (^)

- hasil 1 kalau bit berbeda (1 dan 0)
- kalau sama → 0

```
10 = 01010
29 = 11101
-----------XOR
     10111 = 23
```

NOT (~)

- membalik semua bit
- 1 jadi 0, 0 jadi 1

```
~(0101) = 1010
```

Bitwise Operator

Shift Right (>>)

semua bit digeser ke kanan

```
7 = 111
7 >> 2 → geser 2 kali
→ 001 = 1
```

Shift Left (<<)

semua bit digeser ke kiri

```
9 = 1001
9 << 3 → geser 3 kali
→ 1001000 = 72
```

Special Operator

Ternary (?) Operator

- Ternary operator ? Takes general form:

  Expression 1 ? Expression 2: Expression 3

example:

y = x>9 ? 100:200;

if x = 10 or greater, then value 100 is assigned to y.

if x = 9 or lesser, then value 200 is assigned to y.

- Equivalence code can be written using if-else statement:

```
if (x>9) then
        y=100;
else
        y=200;
```

Special Operator

Operator & (address-of)

& dipakai untuk mengambil alamat memori dari suatu variabel.

example:

```
int x = 10;
printf("%p", &x);   // tampilkan alamat memori x
```

&x = alamat memori variabel x (misalnya 0x7ffe...)

Special Operator

Operator * (dereference)

*** dipakai untuk mengambil nilai yang ada di alamat memori yang ditunjuk pointer.**

example:

```
int x = 10;
int *p; // deklarasi pointer p
p = &x; // simpan alamat x ke pointer p
printf("%d", *p);  // akses nilai di alamat p
```

***p : "nilai dari alamat yang ditunjuk p", yaitu isi x (10)**

Special Operator

Comma Operator

- Comma Operator (,) dipakai untuk menuliskan beberapa ekspresi dalam satu statement
- Nilai hasil akhirnya adalah ekspresi paling kanan
- Dipakai saat inisialisasi dalam for loop

example:

```
x = (y=3, y+1);
urutannya:
y = 3;
y+1; → dihitung nilainya = 4
x = 4 → hasil dari ekspresi kanan (4) dimasukkan ke x
```

Special Operator

Dot (.) and Arrow (->) Operator

Dot (.)

- **Dipakai untuk mengakses bagian (field) dari sebuah structure langsung. Misalnya ada struct Mahasiswa dengan data nama, umur. Kalau variabelnya m1, maka ditulis:**
- **m1.nama → ambil nama mahasiswa**
- **m1.umur → ambil umur mahasiswa**

Special Operator

Dot (.) and Arrow (->) Operator

Arrow (->)

- **Dipakai untuk mengakses struct lewat pointer (alamat memori). Jadi ka jika ada pointer ptr yang menunjuk ke m1, maka ditulis:**
- **ptr->nama**
- **ptr->umur**

Special Operator

[] and () Operator

Square Brackets []

- Dipakai untuk array indexing, yaitu mengambil elemen tertentu dari sebuah array.

Parentheses ()

1. Mengatur prioritas (precedence) dalam ekspresi matematika.

- Tanpa kurung: 2 + 3 * 4 = 14 (karena * lebih dulu dari +)
- Dengan kurung: (2 + 3) * 4 = 20

1. Memanggil fungsi dalam C.

- printf("Halo\n"); // memanggil fungsi printf

Special Operator

Operator Precedence

highest

lowest

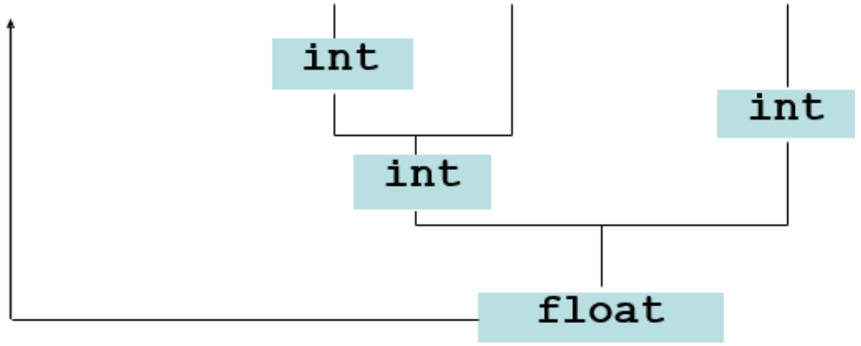
Operators	Associativity
() [] -> .	left to right
! ~ ++ -- + - * (*type*) sizeof	right to left
* / %	left to right
+ -	left to right
<< >>	left to right
< <= > >=	left to right
== !=	left to right
&	left to right
^	left to right
|	left to right
&&	left to right
||	left to right
?:	right to left
= += -= *= /= %= &= ^= |= <<= >>=	right to left
,	left to right

Special Operator

Type ConversionExample

```
char ch;
int i;
float result;

result = (ch / i)  + ch;
```



example:

```
char ch = 'A'; // ASCII 65
int i = 10;
float result;

result = (ch/i) + ch;
```

```
ch/i = 65/10 = 6
(ch/i)+ch = 6 + 65 = 71
result (float)→ 71.0
```

Special Operator

Cast

- cara untuk memaksa sebuah nilai/ekspresi menjadi tipe data tertentu.
- general form: (type) expression

example:

```
int x = 5, y = 2;
float hasil;

hasil = (float)x / y;
```

Kalau tanpa casting:

- x / y = 5 / 2 = 2 (karena int / int hasilnya int, pecahan dibuang).
- Dengan casting (float)x:
- 5.0 / 2 = 2.5 (karena salah satu operand float, hasilnya float).

THANK YOU!